

# 编译原理-词法分析

## 递归下降法与预测分析法

授课教师：柯嵩宇

福州大学计算机与大数据学院

2025 年 3 月 27 日



福州大学  
FUZHOU UNIVERSITY 1958



- ① 课前复习
- ② 自顶向下之递归下降法
- ③ 自顶向下之预测分析法



## ① 课前复习

## ② 自顶向下之递归下降法

## ③ 自顶向下之预测分析法

# 文法

20 世纪 50 年代，诺姆·乔姆斯基首次提出了生成语法的经典形式化理论，其中文法  $G$  由以下部分组成：

- 有限的非终结符号集  $N$ ，与  $G$  生成的字符串无交。
- 有限的终结符号集  $\Sigma$ ，与  $N$  无交。
- 有限的产生式规则集  $P$ ，每个规则都为如下形式

$$(\Sigma \cup N)^* N (\Sigma \cup N)^* \rightarrow (\Sigma \cup N)^*$$

每一个产生式规则都必须保证左边至少有一个非终结符号。

- 开始符号  $S \in N$ 。

文法  $G$  也可以被记作四元组  $(N, \Sigma, P, S)$ 。

# 文法的扩展知识

- 语言  $\{a^n b^n c^n | n \geq 1\}$  的文法:

$S \rightarrow abc$

$S \rightarrow aBSc$

$Ba \rightarrow aB$

$Bb \rightarrow bb$

# 文法的扩展知识

- 语言  $\{a^n b^n c^n | n \geq 1\}$  的文法:

$S \rightarrow abc$

$S \rightarrow aBSc$

$Ba \rightarrow aB$

$Bb \rightarrow bb$

- 从  $S$  推导  $aabbcc$  的过程

$S$

# 文法的扩展知识

- 语言  $\{a^n b^n c^n | n \geq 1\}$  的文法:

$S \rightarrow abc$

$S \rightarrow aBSc$

$Ba \rightarrow aB$

$Bb \rightarrow bb$

- 从  $S$  推导  $aabbcc$  的过程

$S$

# 文法的扩展知识

- 语言  $\{a^n b^n c^n | n \geq 1\}$  的文法:

$S \rightarrow abc$

$S \rightarrow aBSc$

$Ba \rightarrow aB$

$Bb \rightarrow bb$

- 从  $S$  推导  $aabbcc$  的过程

$S \Rightarrow aBSc$

# 文法的扩展知识

- 语言  $\{a^n b^n c^n | n \geq 1\}$  的文法:

$S \rightarrow abc$

$S \rightarrow aBSc$

$Ba \rightarrow aB$

$Bb \rightarrow bb$

- 从  $S$  推导  $aabbcc$  的过程

$S \Rightarrow aBSc \Rightarrow aBabcc$

# 文法的扩展知识

- 语言  $\{a^n b^n c^n | n \geq 1\}$  的文法:

$S \rightarrow abc$

$S \rightarrow aBSc$

$Ba \rightarrow aB$

$Bb \rightarrow bb$

- 从  $S$  推导  $aabbcc$  的过程

$S \Rightarrow aBSc \Rightarrow aBabcc \Rightarrow aaBbcc$

# 文法的扩展知识

- 语言  $\{a^n b^n c^n | n \geq 1\}$  的文法:

$S \rightarrow abc$

$S \rightarrow aBSc$

$Ba \rightarrow aB$

$Bb \rightarrow bb$

- 从  $S$  推导  $aabbcc$  的过程

$S \Rightarrow aBSc \Rightarrow aBabcc \Rightarrow aaBbcc \Rightarrow aabbcc$

- 这个文法是一个典型的上下文相关文法。

# 词法分析

## 正则文法

正则文法  $\mathcal{G} = (\mathbb{N}, \Sigma, \mathbb{P}, S)$ ，其中文法

- 产生式规则集  $\mathbb{P}$ ，每个规则都为如下形式

$$\mathbb{N} \rightarrow \Sigma^* \mathbb{N},$$

即正则文法的产生式左侧仅允许存在一个非终结符，右侧终结符必须在非终结符前面。

- 不是一般性地，我们可以要求只有一个非终结符在右侧的产生式中。

# 词法分析

## 作业讲评

### 例

写一个正则表达式，这个正则表达式能够匹配所有 ab 构成的串中恰好包含偶数个 a 奇数个 b 的。

# 词法分析

## 作业讲评

我们考虑从左往右的第一个  $b$  的情形：

- ① 第一个  $b$  有偶数个  $a$ ，那么后续串中  $ab$  均为偶数个；
- ② 第一个  $b$  有奇数个  $a$ ，则后续串中  $a$  均为奇数个， $b$  为偶数个；

因此，我们首先构造  $ab$  均为偶数的子串  $R$ ，有

$$R = ((aa|bb)|(ab|ba)(aa|bb)^*(ab|ba))^*.$$

那么根据上述讨论，两种情形的对应的正则表达式分别是：

- ①  $(aa)^*bR$ ;
- ②  $(aa)^*abRaR$ ;

在此基础上，提取左右公因式得  $(aa)^*(b|abRa)R$

# 上下文无关文法

上下文无关文法  $\mathcal{G} = (\mathbb{N}, \Sigma, \mathbb{P}, S)$ ，其中文法

- 产生式规则集  $\mathbb{P}$ ，每个规则都为如下形式

$$\mathbb{N} \rightarrow (\Sigma \cup \mathbb{N})^*,$$

即上下文无关文法的产生式左侧仅允许存在一个非终结符。

# 下推自动机

下推自动机 (Pushdown Automaton, PDA) 是一个抽象的机器用于识别和接受字符输入。它在非确定型有限自动机的基础上添加了一个无限规模的栈。它的数学定义为一个六元组:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$$

- $Q$  是所有状态的集合, 集合的大小有限。
- $\Sigma$  是输入字符的集合, 集合的大小有限。特别地,  $\Sigma_\epsilon = \{\epsilon\} \cup \Sigma$
- $\Gamma$  是栈字符的集合, 大小有限。特别地,  $\Gamma_\epsilon = \{\epsilon\} \cup \Gamma$
- $\delta$  是转移函数

$$\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow Q \times \Gamma_\epsilon$$

即同时根据输入和当前栈顶元素进行转移, 在改变状态的同时替换栈顶元素。

- $q_0, F$  的意思和 NFA 中的一致。

# 下推自动机与上下文无关文法的关系

## 定理

一个语言是上下文无关的，当且仅当存在一个下推自动机识别该语言。

# 下推自动机与上下文无关文法的关系

## 定理

一个语言是上下文无关的，当且仅当存在一个下推自动机识别该语言。

## 引理

若一个语言是上下文无关的，则存在一个下推自动机识别能够识别该语言。

# 下推自动机与上下文无关文法的关系

## 定理

一个语言是上下文无关的，当且仅当存在一个下推自动机识别该语言。

## 引理

若一个语言是上下文无关的，则存在一个下推自动机识别能够识别该语言。

## 引理

若一个语言可以被  $PDA$  接受，它便是一个上下文无关语言。

# 下推自动机与上下文无关文法

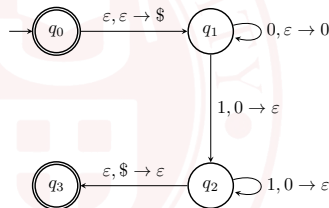
上下文无关文法

$$S \rightarrow 0S1 | \varepsilon$$

上下文无关语言

$$\{0^n 1^n | n \geq 0\}$$

PDA



# 下推自动机与上下文无关文法

## 上下文无关文法

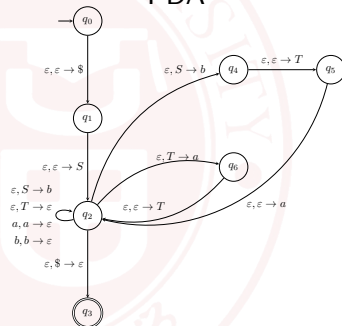
$$S \rightarrow aTb|b$$

$$T \rightarrow Ta|\epsilon$$

## 上下文无关语言

$$\{a^n b | n \geq 0\}$$

## PDA



# 语法分析器的类型

## 自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从根开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

## 自底向上 (Bottom-Up)

- 从输入串开始，逐步进行归约，直到文法的开始符号。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。
- 从叶节点开始，构造语法树。
- 具体实现：算符优先分析法、LR 分析法。

# Top-Down 分析器挑战一：左递归

在自顶向下分析器寻找匹配的推导时，通常采用的是最左推导的形式，即每次选择**最左边**的非终结符进行替换。

按顺序尝试非终结符的不同产生式，有可能出现最左侧符号不断出现的**无限循环**的情况。

文法

目标句子：8+8+8

$expr \rightarrow expr + term \mid expr - term \mid term$

$term \rightarrow term * factor \mid term / factor \mid factor$

$factor \rightarrow digit \mid (expr)$

$digit \rightarrow 0 \mid 1 \mid \dots \mid 9$

$expr \Rightarrow expr + term$

$\Rightarrow expr + term + term$

$\Rightarrow expr + term + term$

$\Rightarrow expr + term + term + term$

$\Rightarrow expr + term + term + term + term$

# Top-Down 分析器挑战一：左递归

左递归的两种形式：

直接左递归

$$\text{expr} \rightarrow \text{expr} + \text{term} | \text{expr} - \text{term} | \text{term}$$
$$\text{term} \rightarrow \text{term} * \text{factor} | \text{term} / \text{factor} | \text{factor}$$
$$\text{factor} \rightarrow \text{digit} | (\text{expr})$$
$$\text{digit} \rightarrow 0 | 1 | \dots | 9$$

间接左递归

$$S \rightarrow AB$$
$$A \rightarrow TC$$
$$T \rightarrow S\gamma$$

# 间接左递归的消除

## Algorithm 1: 消除间接左递归

**Input:** 按照某个顺序将非终结符号排序为  $A_1, A_2, \dots, A_n$

**Result:** 无简介左递归的文法

```
1 for  $i \leftarrow 1$  to  $n$  do
2   for  $j \leftarrow 1$  to  $i-1$  do
3     消除由  $A_j$  导致的左递归, 即向每个形如  $A_i \rightarrow A_j \gamma$  的产生式代入所有  $A_j$ 
      为头的规则, 得到  $A_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$ , 其中  $A_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ , 是所有的
       $A_j$  产生式;
      // 最坏情况会产生  $\mathcal{O}(n \times m)$  个新的产生式
4   end
5   消除  $A_i$  的直接左递归;
      // 方法类似, 同样的有可能产生  $\mathcal{O}(n \times m)$  个产生式
6 end
```

## Top-Down 分析器挑战二：回溯

- 分析过程中，当一个非终结符用某一个候选式匹配成功时，这种匹配可能是暂时的。出错时，需要进行“回溯”。
- 当同一个非终结符的多个候选式存在共同前缀时，将导致回溯。

# Top-Down 分析器挑战二：回溯

## 消除回溯

- 目标：对文法的任何非终结符，当要它去匹配输入串时，能够根据它所面临的输入符号准确地指派它的一个候选去执行任务，并且此候选的工作结果应是确信无疑的。
- 方法：提取左公因子

## Top-Down 分析器挑战二：回溯

如下文法该选择哪条规则？

$$\begin{aligned} stmt &\rightarrow if(expr) \text{ then } stmt \text{ else } stmt \\ &| if(expr) \text{ then } stmt \end{aligned}$$

## Top-Down 分析器挑战二：回溯

如下文法该选择哪条规则？

$$\begin{aligned} stmt &\rightarrow \underline{if (expr) then stmt else stmt} \\ &\quad | \quad \underline{if (expr) then stmt} \end{aligned}$$

手段：通过改写产生式来推迟决定，等读入了足够多的输入，获得足够信息后再做出正确的选择。

$$\begin{aligned} stmt &\rightarrow if (expr) then stmt else stmt \\ elsestmt &\rightarrow \varepsilon \quad | \quad else stmt \end{aligned}$$

# Top-Down 分析器挑战二：回溯

算法：提取左公因式

---

## Algorithm 2: 提取左公因式

---

**Input:** 按照某个顺序将非终结符号排序为  $A_1, A_2, \dots, A_n$

**Result:** 无左公因式的文法

- 1 对于每个非终结符  $A$ ，找出它的两个或多个候选式的最长公共前缀  $\alpha$ 。若  $\alpha \neq \varepsilon$ ，则将所有  $A$ -产生式  $A \rightarrow \alpha\beta_1|\alpha\beta_2|\dots|\alpha\beta_n|\gamma_1|\gamma_2|\dots|\gamma_m$  改写为

$$\begin{aligned} A &\rightarrow \alpha A' |\gamma_1| \gamma_2 | \dots | \gamma_m \\ A' &\rightarrow \beta_1 | \beta_2 | \dots | \beta_n \end{aligned}$$

其中， $\gamma_i$  表示所有不以  $\alpha$  开头的产生式体；

- 2 不断应用这个转换，直到每个非终结符的任意两个产生式体都没有公共前缀为止；

# 提取左公因式

## 例

改写下列文法以便无回溯的自顶向下分析：

$$\begin{aligned} S &\rightarrow cAd \\ A &\rightarrow ab|a. \end{aligned}$$

## 解

对  $A$  提取左公因子得

$$\begin{aligned} S &\rightarrow cAd \\ A &\rightarrow aA' \\ A' &\rightarrow b|\epsilon \end{aligned}$$

# 提取左公因式

## 例

改写文法  $S \rightarrow abcd|abce|abf$  以便无回溯的自顶向下分析。

## 解

$S$  所有子式的最长左公因子为  $ab$ , 引入  $A$  改写生成式得  $S \rightarrow abA, A \rightarrow cd|ce|f$ 。  
再提取  $A$  的左公因子并引入  $B$  改写生成式得  $A \rightarrow cB|f, B \rightarrow d|e$ 。  
最终文法为

$$S \rightarrow abA$$

$$A \rightarrow cB|f$$

$$B \rightarrow d|e$$

# 文法处理的顺序

## ① 消除左递归;



# 文法处理的顺序

- ① 消除左递归;
- ② 提取左公因子以消除回溯;



# 文法处理的顺序

- ① 消除左递归;
- ② 提取左公因子以消除回溯;
- ③ 处理文法的二义性;

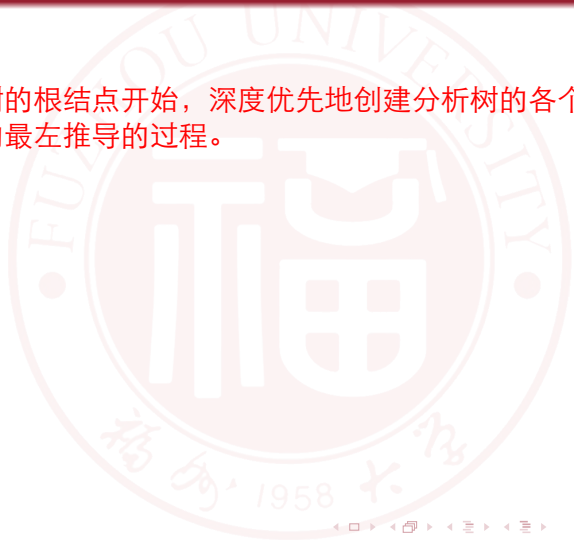




- ① 课前复习
- ② 自顶向下之递归下降法
- ③ 自顶向下之预测分析法

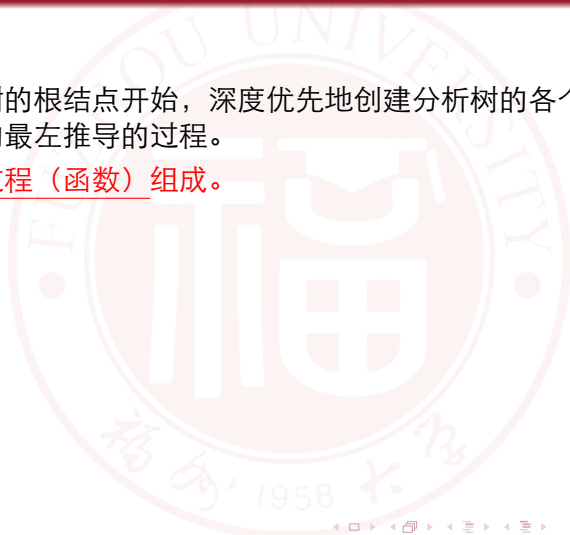
# 递归下降分析法

- 自顶向下的语法分析：从分析树的根结点开始，深度优先地创建分析树的各个结点。可以看作是寻找输入串的最左推导的过程。



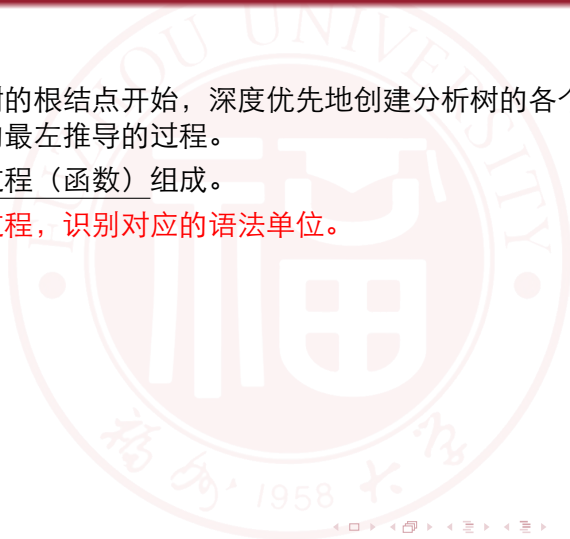
# 递归下降分析法

- 自顶向下的语法分析：从分析树的根结点开始，深度优先地创建分析树的各个结点。可以看作是寻找输入串的最左推导的过程。
- 一个递归下降分析程序由一组过程（函数）组成。



# 递归下降分析法

- 自顶向下的语法分析：从分析树的根结点开始，深度优先地创建分析树的各个结点。可以看作是寻找输入串的最左推导的过程。
- 一个递归下降分析程序由一组过程（函数）组成。
- 每个非终结符号有一个对应的过程，识别对应的语法单位。



# 递归下降分析法

- 自顶向下的语法分析：从分析树的根结点开始，深度优先地创建分析树的各个结点。可以看作是寻找输入串的最左推导的过程。
- 一个递归下降分析程序由一组过程（函数）组成。
- 每个非终结符号有一个对应的过程，识别对应的语法单位。
- 当需要从某个非终结符出发进行展开（推导）时，就调用这个非终结符对应的子程序。

# 递归下降分析法

- 自顶向下的语法分析：从分析树的根结点开始，深度优先地创建分析树的各个结点。可以看作是寻找输入串的最左推导的过程。
- 一个递归下降分析程序由一组过程（函数）组成。
- 每个非终结符号有一个对应的过程，识别对应的语法单位。
- 当需要从某个非终结符出发进行展开（推导）时，就调用这个非终结符对应的子程序。
- 程序的执行从开始符号对应的过程开始，如果这个过程的过程体扫描了整个输入串，它就停止执行并宣布语法分析成功。

# 递归下降分析法

- 自顶向下的语法分析：从分析树的根结点开始，深度优先地创建分析树的各个结点。可以看作是寻找输入串的最左推导的过程。
- 一个递归下降分析程序由一组过程（函数）组成。
- 每个非终结符号有一个对应的过程，识别对应的语法单位。
- 当需要从某个非终结符出发进行展开（推导）时，就调用这个非终结符对应的子程序。
- 程序的执行从开始符号对应的过程开始，如果这个过程的过程体扫描了整个输入串，它就停止执行并宣布语法分析成功。
- 通用的递归下降分析技术可能需要回溯，即它可能要重复扫描输入。

# 递归下降分析法

## 典型过程

### Algorithm 3: 处理非终结符号的典型过程

```

1 void A () {
2   选择一个 A 的产生式,  $A \rightarrow X_1, X_2, \dots, X_k$ ;
3   for  $i \leftarrow 1$  to  $k$  do
4     if  $X_i$  是一个非终结符号 then
5       调用过程  $X_i()$ ;
6     else
7       if  $X_i$  等于当前的输入符号  $a$  then
8         继续读取下一个输入符号;
9       else
10        处理错误;
11      end
12    end
13  end

```

## 该方法的问题

- ① 未说明如何选择产生式（不确定），else 应回到该行继续尝试下一个产生式。
- ② 可能需要回溯（降低效率）。
- ③ 可能出现无限循环。





- ① 课前复习
- ② 自顶向下之递归下降法
- ③ 自顶向下之预测分析法

# FIRST 与 FOLLOW

- FIRST 和 FOLLOW 是两个文法相关的函数。
- 在自顶向下与自底向上的分析器构造中都会用到。
- $\text{FIRST}(\alpha)$  被定义为可以由  $\alpha$  推导得到的串首符号的集合，其中  $\alpha$  是任意的文法符号串，具体地
  - $\forall \alpha \in (\mathbb{V}_T \cup \mathbb{V}_N)^+, \text{FIRST}(\alpha) = \{a | \alpha \Rightarrow^* a\beta, a \in \mathbb{V}_T, \beta \in (\mathbb{V}_T \cup \mathbb{V}_N)^*\}$
  - 若  $\alpha \Rightarrow^* \varepsilon$  合法则， $\varepsilon \in \text{FIRST}(\alpha)$
- 如果非终结符  $A$  的所有候选式的首终结符集两两不相交，当要求  $A$  匹配输入串时， $A$  就能根据当前输入符号  $c$ ，选择唯一的正确的候选式，即串首终结符集中包含  $c$  的那个候选式。

# 计算文法符号 $X$ 的 $\text{FIRST}(X)$

## 例

计算下列文法的  $\text{FIRST}$  集:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

# 计算文法符号 $X$ 的 $\text{FIRST}(X)$

## 例

计算下列文法的  $\text{FIRST}$  集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

# 计算文法符号 $X$ 的 $\text{FIRST}(X)$

## 例

计算下列文法的  $\text{FIRST}$  集:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

# 计算文法符号 $X$ 的 $\text{FIRST}(X)$

## 例

计算下列文法的  $\text{FIRST}$  集:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

# 计算文法符号 $X$ 的 $\text{FIRST}(X)$

## 例

计算下列文法的  $\text{FIRST}$  集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

# 计算 FIRST 的算法

---

**Algorithm 4:** 计算语法符号  $X$  的串首集  $\text{FIRST}(X)$  的算法

---

**Input:** 语法符号  $X$

**Output:** 串首集  $\text{FIRST}(X)$

- 1 如果  $X$  是一个终结符号,



# 计算 FIRST 的算法

---

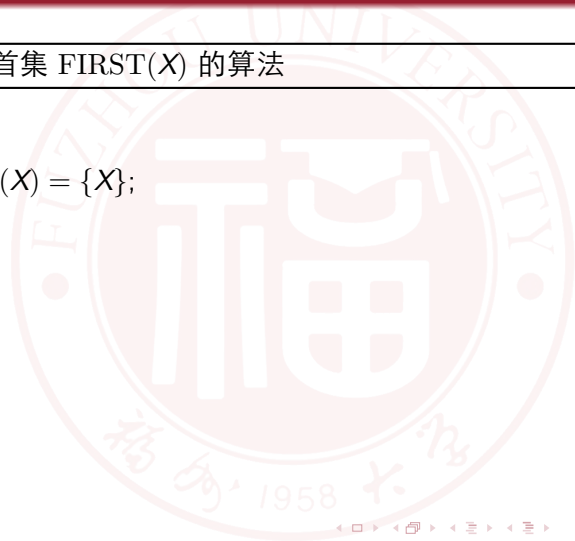
**Algorithm 5:** 计算语法符号  $X$  的串首集  $\text{FIRST}(X)$  的算法

---

**Input:** 语法符号  $X$

**Output:** 串首集  $\text{FIRST}(X)$

- 1 如果  $X$  是一个终结符号, 则  $\text{FIRST}(X) = \{X\}$ ;



# 计算 FIRST 的算法

---

**Algorithm 6:** 计算语法符号  $X$  的串首集  $\text{FIRST}(X)$  的算法

---

**Input:** 语法符号  $X$

**Output:** 串首集  $\text{FIRST}(X)$

- 1 如果  $X$  是一个终结符号, 则  $\text{FIRST}(X) = \{X\}$ ;
- 2 如果  $X$  是一个非终结符号,

# 计算 FIRST 的算法

## Algorithm 7: 计算语法符号 $X$ 的串首集 $\text{FIRST}(X)$ 的算法

**Input:** 语法符号  $X$

**Output:** 串首集  $\text{FIRST}(X)$

- 1 如果  $X$  是一个终结符号, 则  $\text{FIRST}(X) = \{X\}$ ;
- 2 如果  $X$  是一个非终结符号, 则先令  $\text{FIRST}(X) \leftarrow \Phi$ ;
- 3 然后选定一个产生式  $X \rightarrow Y_1 Y_2 \dots Y_k$ , 其中  $k \geq 1$ ;
- 4 **for**  $i \leftarrow 1$  **to**  $k$  **do**
  - 5     如果  $a \in \text{FIRST}(Y_i)$  且  $\forall j < i, \varepsilon \in \text{FIRST}(Y_j)$ , 则  
 $\text{FIRST}(X) \leftarrow \text{FIRST}(X) \cup \{a\}$ ;
- 6 **end**
- 7 如果  $\forall i \geq k, \varepsilon \in \text{FIRST}(Y_i)$ , 则  $\text{FIRST}(X) \leftarrow \text{FIRST}(X) \cup \{\varepsilon\}$ ;
- 8 依次处理  $X$  的所有生成式, 对所有可能的  $\text{FIRST}(X)$  取并集;

# FIRST 与 FOLLOW

- FIRST 和 FOLLOW 是两个文法相关的函数。
- 在自顶向下与自底向上的分析器构造中都会用到。



# FIRST 与 FOLLOW

- FIRST 和 FOLLOW 是两个文法相关的函数。
- 在自顶向下与自底向上的分析器构造中都会用到。
- 对于非终结符号  $A$ , FOLLOW 被定义为可能在某些句型中紧跟在  $A$  右边的终结符号的集合。

## 例

如果存在  $S \Rightarrow^* \alpha A a \beta$  的推导, 则  $a \in \text{FOLLOW}(A)$ 。

# FIRST 与 FOLLOW

- FIRST 和 FOLLOW 是两个文法相关的函数。
- 在自顶向下与自底向上的分析器构造中都会用到。
- 对于非终结符号  $A$ , FOLLOW 被定义为可能在某些句型中紧跟在  $A$  右边的终结符号的集合。

## 例

如果存在  $S \Rightarrow^* \alpha A a \beta$  的推导, 则  $a \in \text{FOLLOW}(A)$ 。

# FIRST 与 FOLLOW

- FIRST 和 FOLLOW 是两个文法相关的函数。
- 在自顶向下与自底向上的分析器构造中都会用到。
- 对于非终结符号  $A$ ，FOLLOW 被定义为可能在某些句型中紧跟在  $A$  右边的终结符号的集合。
- $A$  与  $a$  之间可能存在其他的文法符号，但哪些文法符号都会随着推导被  $\varepsilon$  替换。

## 例

如果存在  $S \Rightarrow^* \alpha A a \beta$  的推导，则  $a \in \text{FOLLOW}(A)$ 。

# FIRST 与 FOLLOW

- FIRST 和 FOLLOW 是两个文法相关的函数。
- 在自顶向下与自底向上的分析器构造中都会用到。
- 对于非终结符号  $A$ , FOLLOW 被定义为可能在某些句型中紧跟在  $A$  右边的终结符号的集合。
- $A$  与  $a$  之间可能存在其他的文法符号, 但哪些文法符号都会随着推导被  $\epsilon$  替换。
- 特别地, 如果  $A$  是某些句型最右的符号, 则  $\$ \in \text{FOLLOW}(A)$ , 其中  $\$$  表示输入的终止 (EOF)。

## 例

如果存在  $S \Rightarrow^* \alpha A a \beta$  的推导, 则  $a \in \text{FOLLOW}(A)$ 。

# 计算 FOLLOW 的算法

---

## Algorithm 8: 计算语法符号 $A$ 的紧随集 $\text{FOLLOW}(A)$ 的算法

---

**Input:** 语法符号  $A$

**Output:** 紧随集  $\text{FOLLOW}(A)$

- 1 将  $\$$  放到  $\text{FOLLOW}(S)$  中;
  - 2 如果存在一个产生式  $A \rightarrow \alpha B \beta$ , 则  $\text{FIRST}(\beta)$  中除了  $\epsilon$  意外的符号都在  $\text{FOLLOW}(B)$  中;
  - 3 如果存在一个产生式  $A \rightarrow \alpha B$ , 或存在  $A \rightarrow \alpha B \beta \wedge \epsilon \in \text{FIRST}(\beta)$ , 则  $\text{FOLLOW}(A)$  所有符号都在  $\text{FOLLOW}(B)$  中;
  - 4 上述步骤重复应用, 知道没有任何新的终结符号可以加入任何非终结符的  $\text{FOLLOW}$  集合中。
-

# 计算文法符号 $X$ 的 FOLLOW( $X$ )

## 例

计算下列文法的 FOLLOW 集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \quad \}$$

$$\text{FOLLOW}(E') = \{ \quad \}$$

$$\text{FOLLOW}(T) = \{ \quad \}$$

$$\text{FOLLOW}(T') = \{ \quad \}$$

$$\text{FOLLOW}(F) = \{ \quad \}$$

# 计算文法符号 $X$ 的 FOLLOW( $X$ )

## 例

计算下列文法的 FOLLOW 集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$, ) \}$$

$$\text{FOLLOW}(E') = \{ \quad \}$$

$$\text{FOLLOW}(T) = \{ \quad \}$$

$$\text{FOLLOW}(T') = \{ \quad \}$$

$$\text{FOLLOW}(F) = \{ \quad \}$$

# 计算文法符号 $X$ 的 FOLLOW( $X$ )

## 例

计算下列文法的 FOLLOW 集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$, ) \}$$

$$\text{FOLLOW}(E') = \{ \$, ) \}$$

$$\text{FOLLOW}(T) = \{ \quad \}$$

$$\text{FOLLOW}(T') = \{ \quad \}$$

$$\text{FOLLOW}(F) = \{ \quad \}$$

# 计算文法符号 $X$ 的 FOLLOW( $X$ )

## 例

计算下列文法的 FOLLOW 集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$, ) \}$$

$$\text{FOLLOW}(E') = \{ \$, ) \}$$

$$\text{FOLLOW}(T) = \{ + \}$$

$$\text{FOLLOW}(T') = \{ \}$$

$$\text{FOLLOW}(F) = \{ \}$$

# 计算文法符号 $X$ 的 FOLLOW( $X$ )

## 例

计算下列文法的 FOLLOW 集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$, ) \}$$

$$\text{FOLLOW}(E') = \{ \$, ) \}$$

$$\text{FOLLOW}(T) = \{ +, \$, ) \}$$

$$\text{FOLLOW}(T') = \{ \quad \}$$

$$\text{FOLLOW}(F) = \{ \quad \}$$

# 计算文法符号 $X$ 的 FOLLOW( $X$ )

## 例

计算下列文法的 FOLLOW 集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$, ) \}$$

$$\text{FOLLOW}(E') = \{ \$, ) \}$$

$$\text{FOLLOW}(T) = \{ +, \$, ) \}$$

$$\text{FOLLOW}(T') = \{ +, \$, ) \}$$

$$\text{FOLLOW}(F) = \{ \quad \quad \quad \}$$

# 计算文法符号 $X$ 的 FOLLOW( $X$ )

## 例

计算下列文法的 FOLLOW 集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$, ) \}$$

$$\text{FOLLOW}(E') = \{ \$, ) \}$$

$$\text{FOLLOW}(T) = \{ +, \$, ) \}$$

$$\text{FOLLOW}(T') = \{ +, \$, ) \}$$

$$\text{FOLLOW}(F) = \{ *, ) \}$$

# 计算文法符号 $X$ 的 FOLLOW( $X$ )

## 例

计算下列文法的 FOLLOW 集：

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \varepsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT' | \varepsilon$$

$$F \rightarrow (E) | id$$

$$\text{FIRST}(E) = \{ (, id \}$$

$$\text{FIRST}(E') = \{ +, \varepsilon \}$$

$$\text{FIRST}(T) = \{ (, id \}$$

$$\text{FIRST}(T') = \{ *, \varepsilon \}$$

$$\text{FIRST}(F) = \{ (, id \}$$

$$\text{FOLLOW}(E) = \{ \$, ) \}$$

$$\text{FOLLOW}(E') = \{ \$, ) \}$$

$$\text{FOLLOW}(T) = \{ +, \$, ) \}$$

$$\text{FOLLOW}(T') = \{ +, \$, ) \}$$

$$\text{FOLLOW}(F) = \{ *, +, \$, ) \}$$

## 练习

计算下列文法的 FIRST 和 FOLLOW 集:

$$S \rightarrow AB|bC$$

$$A \rightarrow \varepsilon|b$$

$$B \rightarrow \varepsilon|aD$$

$$C \rightarrow AD|b$$

$$D \rightarrow aS|c$$

## 练习

计算下列文法的 FIRST 和 FOLLOW 集:

$$S \rightarrow AB|bC$$

$$A \rightarrow \varepsilon|b$$

$$B \rightarrow \varepsilon|aD$$

$$C \rightarrow AD|b$$

$$D \rightarrow aS|c$$

$$\text{FIRST}(S) = \{\varepsilon, a, b\}$$

$$\text{FIRST}(A) = \{\varepsilon, b\}$$

$$\text{FIRST}(B) = \{\varepsilon, a\}$$

$$\text{FIRST}(C) = \{a, b, c\}$$

$$\text{FIRST}(D) = \{a, b\}$$

若  $A \rightarrow \alpha B \beta \in \mathbb{P}$ , 则  $\text{FIRST}(\beta) / \{\varepsilon\} \subseteq \text{FOLLOW}(B)$ 。

若  $A \rightarrow aB \in \mathbb{P}$  或  $A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \varepsilon \in \text{FIRST}(\beta)$ , 则  $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$ 。

## 练习

计算下列文法的 FIRST 和 FOLLOW 集：

$$S \rightarrow AB|bC$$

$$A \rightarrow \varepsilon|b$$

$$B \rightarrow \varepsilon|aD$$

$$C \rightarrow AD|b$$

$$D \rightarrow aS|c$$

$$\text{FIRST}(S) = \{\varepsilon, a, b\}$$

$$\text{FIRST}(A) = \{\varepsilon, b\}$$

$$\text{FIRST}(B) = \{\varepsilon, a\}$$

$$\text{FIRST}(C) = \{a, b, c\}$$

$$\text{FIRST}(D) = \{a, b\}$$

若  $A \rightarrow \alpha B \beta \in \mathbb{P}$ , 则  $\text{FIRST}(\beta)/\{\varepsilon\} \subseteq \text{FOLLOW}(B)$ 。

若  $A \rightarrow aB \in \mathbb{P}$  或  $A \rightarrow \alpha B \beta \in \mathbb{P} \wedge \varepsilon \in \text{FIRST}(\beta)$ , 则  $\text{FOLLOW}(A) \subseteq \text{FOLLOW}(B)$ 。

## 练习

计算下列文法的 FIRST 和 FOLLOW 集：

$$S \rightarrow AB|bC$$

$$A \rightarrow \varepsilon|b$$

$$B \rightarrow \varepsilon|aD$$

$$C \rightarrow AD|b$$

$$D \rightarrow aS|c$$

$$\text{FIRST}(S) = \{\varepsilon, a, b\}$$

$$\text{FIRST}(A) = \{\varepsilon, b\}$$

$$\text{FIRST}(B) = \{\varepsilon, a\}$$

$$\text{FIRST}(C) = \{a, b, c\}$$

$$\text{FIRST}(D) = \{a, b\}$$

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \{\$, a, c\}$$

$$\text{FOLLOW}(B) = \{\$ \}$$

$$\text{FOLLOW}(C) = \{\$ \}$$

$$\text{FOLLOW}(D) = \{\$ \}$$

# LL(1) 文法

## 定义

- LL(1) 文法是一类特别的上下文无关文法。



# LL(1) 文法

## 定义

- LL(1) 文法是一类特别的上下文无关文法。
  - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；

# LL(1) 文法

## 定义

- LL(1) 文法是一类特别的上下文无关文法。
  - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
  - 第二个 L 表示 “Leftmost”，即最左推导；

# LL(1) 文法

## 定义

- LL(1) 文法是一类特别的上下文无关文法。
  - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
  - 第二个 L 表示 “Leftmost”，即最左推导；
  - (1) 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。

# LL(1) 文法

## 定义

- LL(1) 文法是一类特别的上下文无关文法。
  - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
  - 第二个 L 表示 “Leftmost”，即最左推导；
  - (1) 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。

# LL(1) 文法

## 定义

- LL(1) 文法是一类特别的上下文无关文法。
  - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
  - 第二个 L 表示 “Leftmost”，即最左推导；
  - (1) 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- 左递归文法和二义性文法都不是 LL(1) 文法。

# LL(1) 文法

## 定义

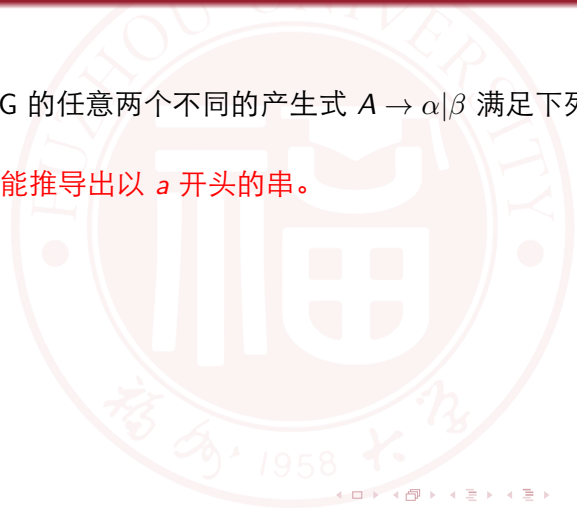
- LL(1) 文法是一类特别的上下文无关文法。
  - 第一个 L 表示 “Left-to-Right”，即从左往右扫描；
  - 第二个 L 表示 “Leftmost”，即最左推导；
  - (1) 表示每次推导只需要看一个输入符号就可以决定语法分析动作，即决定使用那个推导式。
- 虽然在为语言设计文法的时候要多加小心，LL(1) 文法已经足以描述大部分程序设计语言构造。
- 左递归文法和二义性文法都不是 LL(1) 文法。
- LL(1) 文法通常将  $\epsilon$  边作为最后的默认选择以解决  $\epsilon$  的二义性（例如，if 语句的 else 匹配问题）。

# LL(1) 文法

$G$  是 LL(1) 文法的充分必要条件

一个文法  $G$  是 LL(1) 文法当且仅当  $G$  的任意两个不同的产生式  $A \rightarrow \alpha | \beta$  满足下列条件

- ① 不存在终结符号  $a$  使  $\alpha$  和  $\beta$  都能推导出以  $a$  开头的串。



# LL(1) 文法

$G$  是 LL(1) 文法的充分必要条件

一个文法  $G$  是 LL(1) 文法当且仅当  $G$  的任意两个不同的产生式  $A \rightarrow \alpha | \beta$  满足下列条件

- ① 不存在终结符号  $a$  使  $\alpha$  和  $\beta$  都能推导出以  $a$  开头的串。
- ②  $\alpha$ 、 $\beta$  中最多一个可以推导出空串  $\varepsilon$ 。

# LL(1) 文法

$G$  是 LL(1) 文法的充分必要条件

一个文法  $G$  是 LL(1) 文法当且仅当  $G$  的任意两个不同的产生式  $A \rightarrow \alpha | \beta$  满足下列条件

- ① 不存在终结符号  $a$  使  $\alpha$  和  $\beta$  都能推导出以  $a$  开头的串。
- ②  $\alpha$ 、 $\beta$  中最多一个可以推导出空串  $\varepsilon$ 。

# LL(1) 文法

$G$  是 LL(1) 文法的充分必要条件

一个文法  $G$  是 LL(1) 文法当且仅当  $G$  的任意两个不同的产生式  $A \rightarrow \alpha | \beta$  满足下列条件

- ① 不存在终结符号  $a$  使  $\alpha$  和  $\beta$  都能推导出以  $a$  开头的串。
- ②  $\alpha$ 、 $\beta$  中最多一个可以推导出空串  $\varepsilon$ 。  
(上面两个条件等价于  $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ )
- ③ 若  $\beta \Rightarrow^* \varepsilon$ ，则  $\alpha$  不能推导出任何以  $\text{FOLLOW}(A)$  中某个终结符开头的串。反之亦然。

# LL(1) 文法

$G$  是 LL(1) 文法的充分必要条件

一个文法  $G$  是 LL(1) 文法当且仅当  $G$  的任意两个不同的产生式  $A \rightarrow \alpha | \beta$  满足下列条件

- ① 不存在终结符号  $a$  使  $\alpha$  和  $\beta$  都能推导出以  $a$  开头的串。
- ②  $\alpha$ 、 $\beta$  中最多一个可以推导出空串  $\varepsilon$ 。  
(上面两个条件等价于  $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ )
- ③ 若  $\beta \Rightarrow^* \varepsilon$ , 则  $\alpha$  不能推导出任何以  $\text{FOLLOW}(A)$  中某个终结符开头的串。反之亦然。

# LL(1) 文法

$G$  是 LL(1) 文法的充分必要条件

一个文法  $G$  是 LL(1) 文法当且仅当  $G$  的任意两个不同的产生式  $A \rightarrow \alpha | \beta$  满足下列条件

- ① 不存在终结符号  $a$  使  $\alpha$  和  $\beta$  都能推导出以  $a$  开头的串。
- ②  $\alpha$ 、 $\beta$  中最多一个可以推导出空串  $\varepsilon$ 。  
(上面两个条件等价于  $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ )
- ③ 若  $\beta \Rightarrow^* \varepsilon$ , 则  $\alpha$  不能推导出任何以  $\text{FOLLOW}(A)$  中某个终结符开头的串。反之亦然。  
( $\varepsilon \in \text{FIRST}(\beta) \rightarrow \text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$ )

# 表驱动的分析法

- 特征：不需要为每个非终结符编写递归下降过程，而是根据预测分析表构造一个不需要回溯的非递归下降语法分析器。
- 要求：文法是 LL(1) 的。只有对于 LL(1) 文法，可以对输入串进行有效的无回溯的自上而下分析。